



SAPIENZA
UNIVERSITÀ DI ROMA

Oltre il Gold Standard: analisi comparativa tra metriche tradizionali e approcci LLM-as-a-Judge per la valutazione del web parsing

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica
Ingegneria Informatica e Automatica

Ludovica Fiocchetta

Matricola 2103669

Relatore

Professore Roberto Navigli

Anno Accademico 2025/2026

**Oltre il Gold Standard: analisi comparativa tra metriche tradizionali e approcci
LLM-as-a-Judge per la valutazione del web parsing**

Laurea Triennale. Sapienza Università di Roma

© 2026 Ludovica Fiocchetta. Tutti i diritti riservati

Questa tesi è stata composta con $\text{\LaTeX}$ e la classe Sapthesis.

Email dell'autore: fiocchetta.2103669@studenti.uniroma1.it

Indice

1	Introduzione	1
1.1	Contesto generale	1
1.2	Motivazione: il progetto Chat Minerva	2
1.3	Obiettivi e contributo dell'elaborato	2
1.4	Struttura della tesi	3
2	Lavori correlati	5
2.1	Tecniche di estrazione	5
2.2	Metriche tradizionali	5
2.3	Metriche semantiche	5
2.4	LLM-as-a-Judge	5
3	Sistema presentato: architettura e implementazione	7
3.1	Panoramica dell'infrastruttura	7
3.2	Progettazione del backend	8
3.2.1	Gestione degli endpoint	9
3.2.2	Logica dei parser	11
3.2.3	Motore di calcolo delle metriche	13
3.3	Progettazione del frontend	15
3.3.1	Architettura API e gestione delle latenze	15
3.4	Gestione dei dati	18

Capitolo 1

Introduzione

1.1 Contesto generale

Nell'attuale contesto informativo tecnologico, il World Wide Web rappresenta il più vasto bacino di conoscenza non strutturata mai esistito. Con lo sviluppo dei Large Language Models, l'abilità di elaborare e comprendere il linguaggio naturale è diventata una necessità, poiché l'efficacia di questi modelli è fortemente dipendente dalla qualità dei dati forniti in input. Gli LLM hanno ancora una capacità limitata di manipolare con precisione la conoscenza, infatti falliscono nell'accedere a fonti precise e tendono ad avere allucinazioni su compiti *knowledge-intensive*. Le sfide più note includono:

- Presentazione di informazioni false quando non si ha la risposta;
- Presentazione di informazioni obsolete o generiche quando l'utente si aspetta una risposta specifica e attuale;
- Creazione di una risposta da fonti non autorevoli;
- Creazione di risposte imprecise a causa della confusione terminologica, in cui diverse fonti di addestramento utilizzano la stessa terminologia per parlare di argomenti diversi.

Per questi motivi il processo di estrazione di dati puliti assume un ruolo fondamentale. Nel paradigma della *Retrieval-Augmented Generation* (RAG) [1], architettura che ottimizza le prestazioni di un modello di intelligenza artificiale collegandolo a knowledge base esterne, il *web scraping* tradizionale, basato su regole deterministiche e selettori del Document Object Model (DOM), risulta spesso fragile di fronte alla grande diversità delle strutture HTML. Questo approccio spesso lascia insieme al testo utile anche "rumore", come menu di navigazione, banner pubblicitari e riferimenti ad altre pagine, difetto che può tradursi in allucinazioni nelle risposte degli LLM. Un approccio più robusto richiede sistemi di *web parsing* semantico, che consentono di migliorare i risultati essendo in grado di scansionare grandi database e di isolare esclusivamente le informazioni rilevanti nella pagina, ignorando tutti gli elementi di disturbo.

1.2 Motivazione: il progetto Chat Minerva

Questa necessità operativa si manifesta concretamente all'interno del progetto Chat Minerva sviluppato da Sapienza Università di Roma, per il quale è indispensabile fornire al modello testi in input totalmente puliti al fine di garantire risposte attendibili. Chat Minerva è un modello linguistico avanzato che si pone l'obiettivo di fornire un agente di chat affidabile, capace di rispondere in modo accurato e contestualizzato alle molteplici domande da parte degli utenti. Si basa su un'architettura *Retrieval-Augmented Generation* (RAG), così da fare riferimento a una base di conoscenza autorevole al di fuori dei suoi dati di addestramento originali prima di generare una risposta [2], il cui flusso operativo è illustrato in Figura 1.1.

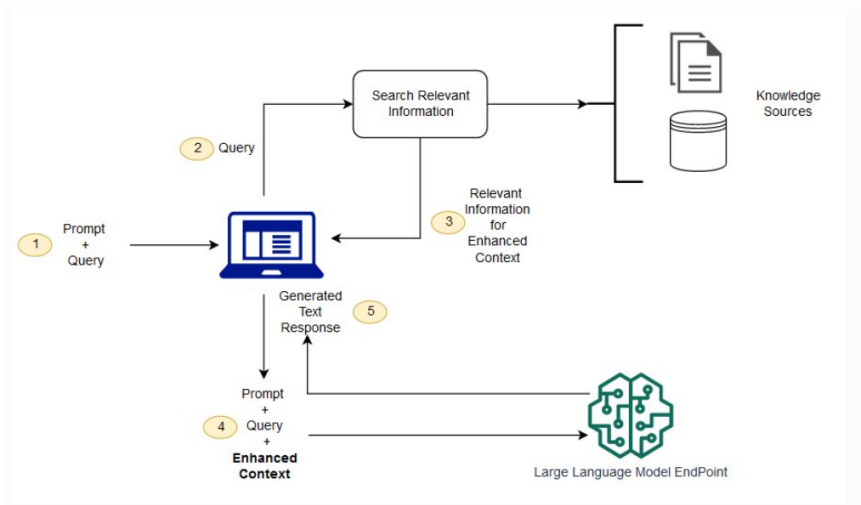


Figura 1.1. Rappresentazione di un'architettura RAG. Il framework sviluppato assicura la qualità dei dati estratti prima del loro inserimento nelle *Knowledge Sources* [2].

Come evidenziato dallo schema, quando l'utente formula una query (passaggi 1 e 2), il sistema ricerca informazioni pertinenti all'interno delle proprie fonti di conoscenza esterne per arricchire il contesto del prompt (passaggi 3 e 4). Se i parser che alimentano questi database estraggono testo sporco o frammenti di codice HTML, l'agente genererà risposte errate o allucinate (passaggio 5). Affinché il modello non generi informazioni sbagliate o non pertinenti è essenziale che i dati forniti in fase di inferenza siano estremamente puliti. Se i parser utilizzati per alimentare Minerva estraggono testo sporco o formattazioni residue, la qualità della risposta del modello degrada drasticamente. È nata quindi la necessità ingegneristica di sviluppare un'infrastruttura capace di testare, misurare e validare in modo oggettivo le performance di diversi sistemi di web parsing, garantendo che al Chatbot venga passata esclusivamente la conoscenza utile e strutturata di cui ha bisogno.

1.3 Obiettivi e contributo dell'elaborato

Il seguente elaborato ha dunque come scopo principale la descrizione di un sistema di estrazione automatizzata di testo pulito da quattro domini internet, e la succes-

siva analisi comparativa sulla valutazione della qualità dei parser. Questo studio metterà a confronto le metriche matematiche tradizionali, che basano il giudizio sulla comparazione con un *gold standard*, con l'innovativo paradigma *LLM-as-a-Judge*, esplorato anche in scenari *reference-free*. Nello specifico, il sistema si articola nei seguenti punti:

- **Sviluppo della pipeline di estrazione e valutazione:** implementazione di un'infrastruttura Python capace di estrarre testo pulito e misurare in modo oggettivo le performance dei sistemi di *web parsing*.
- **Analisi con metriche classiche e avanzate:** valutazione dei *parser* basata sul confronto dei testi con un *gold standard*, calcolando metriche rigorose quali Precision e Recall, ma anche semantiche come BERTScore e METEOR.
- **Sperimentazione del paradigma LLM-as-a-Judge:** introduzione di un approccio moderno in cui un Large Language Model agisce da giudice, esplorando l'efficacia di valutazioni *reference-free* per superare la rigidità delle metriche tradizionali.

Il contributo principale dell'elaborato risiede proprio in questo doppio livello della pipeline di valutazione. Infatti, da un lato si ha un giudizio basato su un *gold standard*, in cui il contenuto estratto viene confrontato con un testo di riferimento salvato all'interno di un database relazionale. In questa fase vengono calcolate metriche classiche basate sulla sovrapposizione insiemistica e sulle distanze operative (come la *Jaccard Similarity*), affiancate da metriche semantiche avanzate (come BERTScore e METEOR). Dall'altro lato, si esplora l'approccio *LLM-as-a-Judge*, chiamando un modello linguistico per valutare la coerenza e la pulizia del testo estratto anche in modalità *reference-free*. Questo permette di superare il limite intrinseco del *gold standard*, che richiede una costosa annotazione manuale preventiva. L'LLM agisce come giudice autonomo: attraverso tecniche di *prompt engineering* e pipeline difensive contro i colli di bottiglia computazionali, è in grado di analizzare la pulizia dell'estrazione anche su domini web sconosciuti, fornendo un punteggio quantitativo e un commento qualitativo pronto all'uso. Questo studio dimostrerà come l'integrazione di approcci semantici avanzati sia essenziale per certificare la qualità del testo estratto, garantendo l'affidabilità di sistemi complessi come Chat Minerva. A supporto di tale studio, l'intero framework software sviluppato è rilasciato in modalità *open-source* ed è liberamente consultabile su GitHub¹.

1.4 Struttura della tesi

La trattazione è organizzata come segue:

- **Capitolo 2:** analizza i lavori correlati, descrivendo l'evoluzione delle tecniche di *web parsing* e presentando il fondamento teorico delle metriche di valutazione impiegate nel progetto, dagli approcci tradizionali e semantici fino all'introduzione del paradigma *LLM-as-a-Judge*.

¹Repository ufficiale del progetto: <https://github.com/ludofiocchetta-glitch/tesi-parser-and-llm-as-a-judge>

- **Capitolo 3:** descrive in dettaglio la progettazione architetturale del sistema, illustrando dettagliatamente tutte le scelte informatiche adottate.
- **Capitolo 4:** esplora l'implementazione degli approcci *LLM-as-a-Judge*, analizzando la costruzione dei *prompt* di valutazione e l'integrazione con i modelli in esecuzione locale.
- **Capitolo 5:** presenta l'impostazione degli esperimenti sui domini selezionati e discute criticamente i risultati ottenuti, confrontando i punteggi delle metriche deterministiche con il giudizio qualitativo dell'intelligenza artificiale.
- **Capitolo 6:** riassume i traguardi raggiunti e delinea i possibili sviluppi futuri per il potenziamento dell'ecosistema.

Capitolo 2

Lavori correlati

2.1 Tecniche di estrazione

2.2 Metriche tradizionali

2.3 Metriche semantiche

2.4 LLM-as-a-Judge

Capitolo 3

Sistema presentato: architettura e implementazione

3.1 Panoramica dell'infrastruttura

Questo capitolo descrive nel dettaglio l'architettura ingegneristica alla base del framework di valutazione. Il sistema ha come obiettivo principale l'estrazione automatizzata di testo pulito dalle pagine di quattro domini internet tramite la libreria *Crawl4AI* e la successiva valutazione dell'accuratezza dei parser, mediante il confronto tra il testo estratto dal *crawler* e il *gold standard* di riferimento creato manualmente. I domini di riferimento sono:

- `en.wikipedia.org`;
- `viaggi-usa.it`;
- `cbsnews.com`;
- `cnbc.com`.

Per gestire in modo efficiente l'estrazione da queste sorgenti, il progetto è stato strutturato seguendo un'architettura a microservizi, separando la logica di estrazione, valutazione e salvataggio dei dati (*Backend*) dall'interfaccia utente (*Frontend*). L'intero sistema è containerizzato mediante Docker, tramite le specifiche definite nel file `docker-compose.yaml` per garantire la portabilità e riproducibilità dell'ambiente di esecuzione. Docker è infatti una piattaforma *open source* per lo sviluppo, la distribuzione e l'esecuzione delle applicazioni. Consente di separare queste ultime dall'infrastruttura grazie all'utilizzo dei *container*, ambienti leggeri e isolati che racchiudono tutto il necessario per eseguire un'applicazione in modo indipendente dal sistema host [3]. Nello specifico del framework, l'orchestrazione tramite Compose ha permesso di avviare simultaneamente e far comunicare i moduli di estrazione, il database relazionale, il modello linguistico locale e l'interfaccia utente. L'intero codice sorgente del progetto è disponibile pubblicamente come anticipato in precedenza su GitHub.

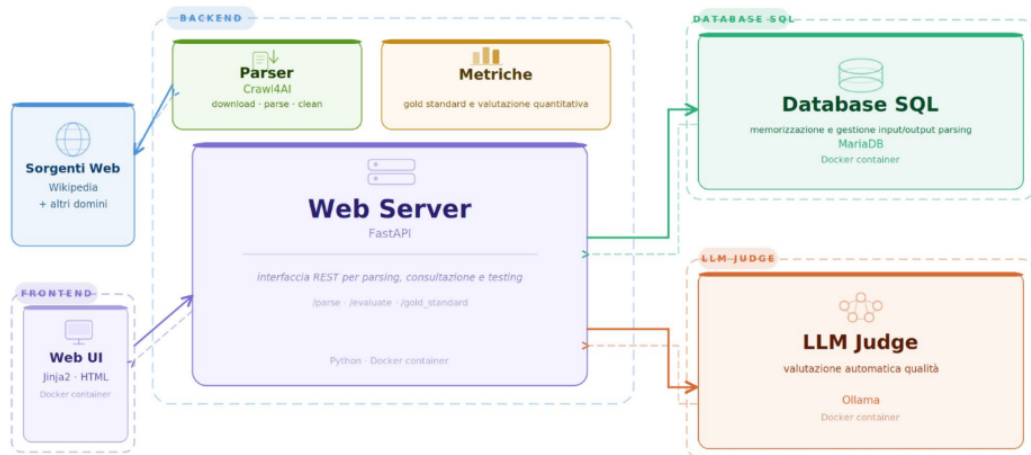


Figura 3.1. Architettura a microservizi del progetto. Lo schema illustra la comunicazione tra le varie componenti del sistema.

3.2 Progettazione del backend

Il *backend*, esposto sulla porta 8003, è stato sviluppato seguendo un'architettura modulare in Python e si articola nei seguenti macro-componenti fondamentali:

- **Validazione dati e server (`server.py`):** questo modulo espone i servizi verso il *frontend* sfruttando il framework FastAPI. Implementa l'interfaccia REST API e utilizza modelli Pydantic per una validazione rigorosa dei dati in ingresso e in uscita, assicurando che ogni endpoint (`/parse`, `/domains`, `/gold_standard`, ecc.) rispetti le specifiche richieste. La logica di *routing* smista dinamicamente le richieste ai diversi *parser* in base al dominio dell'URL fornito.
- **Motore di estrazione:** ognuno dei quattro domini è gestito da un modulo dedicato che implementa una *pipeline* di *parsing* a tre stadi basata sulla libreria Crawl4AI:
 1. *Configurazione e acquisizione:* viene utilizzato `BrowserConfig` in modalità *headless* e `CrawlerRunConfig` con `CacheMode.BYPASS` per garantire l'acquisizione in tempo reale del contenuto. In alternativa, fornendo in input una stringa HTML, permette l'estrazione statica diretta senza interrogazione web.
 2. *DOM cleaning tramite JavaScript:* prima della conversione in Markdown, viene iniettato uno script JS mirato per manipolare il DOM e rimuovere selettivamente elementi di disturbo come infobox, sidebar, footer e banner.
 3. *Post-processing regex:* il testo convertito in Markdown subisce un ulteriore filtraggio tramite espressioni regolari per eliminare residui di tag, metadati, riferimenti bibliografici e spazi bianchi ridondanti, restituendo un testo pulito pronto per l'analisi.

- *Calcolo delle metriche (**accuracy_tester.py**)*: rappresenta il motore di valutazione dell'intero sistema. È stato progettato come un servizio indipendente che riceve stringhe di testo (estratto e di riferimento) e calcola il grado di sovrapposizione tra i set di token, operando una normalizzazione preventiva (*lowercase* e rimozione della punteggiatura) per garantire l'oggettività del confronto.

Completano l'architettura *backend* gli *script* ausiliari `init_db.py` per l'inizializzazione della base di dati, `remove_markdown.py` per la rimozione finale del Markdown e `clean_html.py` per la pulizia dell'html grezzo necessaria all'*LLM-as-a-Judge* in configurazione *Reference-Free*.

3.2.1 Gestione degli endpoint

Il cuore comunicativo del *backend* è gestito da FastAPI, un framework *web* moderno e ad alte prestazioni per la creazione di API in Python, basato sulla tipizzazione standard *type hints*. I principali punti di forza di questo strumento risiedono nella sua robustezza, nella facilità di sviluppo e nella capacità di generare automaticamente una documentazione interattiva [4]. Il framework espone i servizi attraverso un'architettura RESTful e garantisce la validazione di ogni richiesta e risposta tramite i modelli *Pydantic*. Questi ultimi prevengono la creazione di dati malformati imponendo formati specifici (come `ParseRequest` o `EvaluateRequest`) e assicurano la generazione di JSON per una facile integrazione con gli altri componenti del sistema [5].

Le rotte dell'API possono essere raggruppate in tre macro-categorie:

- **Servizi di estrazione (`/parse`)**: l'endpoint principale accetta richieste POST contenenti l'URL da pulire. Il dominio viene analizzato e instradato dinamicamente al *parser* di riferimento per l'elaborazione. Una variabile booleana consente di verificare preliminarmente la presenza dell'HTML nel database locale: se `local=True` il *parsing* avviene off-line, riducendo drasticamente i tempi di latenza e le richieste di rete, altrimenti viene scaricato l'HTML originale.
- **Servizi di valutazione (`/evaluate` e `/evaluate_judge`)**: questi endpoint costituiscono il motore di analisi quantitativa. `/evaluate` esegue la pulizia del Markdown (`remove_markdown`) e restituisce le metriche deterministiche e semantiche calcolate dal modulo `accuracy_tester.py`. Le rotte `/evaluate_judge` e `/evaluate_judge_no_gs` instaurano invece una comunicazione asincrona, tramite la libreria `httpx` [6], con il container di Ollama, interrogando l'LLM con i *prompt* opportunamente ingegnerizzati e imponendo al modello di restituire un output rigorosamente in formato JSON, contenente il voto numerico e il *feedback* qualitativo.
- **Gestione dello stato e statistiche (`/db_stats`, `/full_gs_eval`)**: un set di endpoint dedicati al monitoraggio e all'aggregazione dei dati. Permettono al *frontend* di calcolare le medie delle metriche per ogni dominio, interrogare e salvare i dati nel database e gestire le operazioni CRUD (Create, Read,

Update, Delete) sulle risorse web memorizzate e sui *gold standard*. Completa il modulo la rotta `/status` per il controllo dell'attività del *backend*, dell'integrità del *database* e della raggiungibilità del modello linguistico.

Di seguito è riportato un frammento di codice che mostra i modelli di validazione, inclusa la struttura annidata della risposta e l'endpoint dedicato al calcolo delle metriche di valutazione:

Listing 3.1. Definizione dei modelli Pydantic e dell'endpoint per la valutazione quantitativa.

```

1  # Modello per le metriche standard
2  class TokenLevelEval(BaseModel):
3      precision: float
4      recall: float
5      f1: float
6  # Modello per le altre metriche
7  class XEval(BaseModel):
8      jaccard_similarity: float
9      bigram_overlap: float
10     cosine_similarity: float
11     meteor: float
12     bert_score: float
13
14 # Modelli di Input/Output per l'endpoint /evaluate
15 class EvaluateRequest(BaseModel):
16     parsed_text: str
17     gold_text: str
18
19 class EvaluateResponse(BaseModel):
20     token_level_eval: TokenLevelEval
21     x_eval: XEval
22
23 # Endpoint per il calcolo delle metriche
24 @app.post("/evaluate", response_model=EvaluateResponse)
25 async def evaluate_text(data: EvaluateRequest):
26
27     # Rimozione del Markdown prima della valutazione
28     normalized_parsed = remove_markdown(data.parsed_text)
29     normalized_gold = remove_markdown(data.gold_text)
30
31     # Chiamata al motore di calcolo indipendente
32     ris = calculate_metrics(normalized_parsed, normalized_gold)
33
34     return {
35         "token_level_eval": TokenLevelEval(
36             precision=ris["precision"],
37             recall=ris["recall"],
38             f1=ris["f1"]
39         ),
40         "x_eval": XEval(
41             jaccard_similarity=ris["jaccard_similarity"],
42             bigram_overlap=ris["bigram_overlap"],
43             cosine_similarity=ris["cosine_similarity"],
44             meteor=ris["meteor"],
45             bert_score=ris["bert_score"]
46     )

```

3.2.2 Logica dei parser

La fase di estrazione e pulizia dei dati è affidata a script specializzati per singolo dominio, implementati sfruttando la libreria asincrona *Crawl4AI*. Questo strumento *open-source* è stato ideato per facilitare l'integrazione con i modelli linguistici e le *pipeline* RAG, generando nativamente testo in formato Markdown [7]. Nonostante i selettori CSS e le espressioni regolari varino a seconda del dominio, la logica generale dei *parser* rimane invariata.

L'estrazione si articola in tre fasi sequenziali:

1. **Configurazione e acquisizione:** impostazione del *browser headless* e del generatore Markdown, con regole restrittive per ignorare link e immagini e isolare esclusivamente la classe con i contenuti rilevanti della pagina, al fine di ridurre il rumore.
2. **Manipolazione del DOM:** iniezione di codice JavaScript eseguito direttamente nel *browser* prima dell'estrazione. Questo passaggio è essenziale per rimuovere alla radice elementi strutturali non informativi specifici per ogni dominio (come *sidebar*, *footer* e banner pubblicitari) prima che vengano convertiti in testo.
3. **Filtraggio tramite regex:** raffinamento finale sul testo Markdown generato, per eliminare elementi di disturbo residui come tag di formattazione o metadati non catturati dai selettori CSS.

A titolo esemplificativo, di seguito è riportata l'implementazione del *parser* dedicato al dominio di Wikipedia. Il codice evidenzia la logica di rimozione delle *info-box* tramite JavaScript e la successiva pulizia dei tipici tag redazionali (es. `[edit]`, `[citation needed]`) tramite espressioni regolari.

Listing 3.2. Implementazione del *parser* asincrono per il dominio Wikipedia.

```

1 from crawl4ai import AsyncWebCrawler, BrowserConfig, CrawlerRunConfig
  , CacheMode, DefaultMarkdownGenerator
2 import re
3
4 async def parser_wiki(url: str, html_text: str) -> dict:
5
6     # Configurazione del browser
7     browser_config = BrowserConfig(headless=True)
8
9     # Configurazione del Markdown Generator
10    md_generator = DefaultMarkdownGenerator(
11        options={
12            "ignore_links": True,
13            "escape_html": False,
14            "ignore_images": True
15        }
16    )
17
18    # Iniezione JavaScript per la pulizia del DOM

```

```

19     remove_infobox_js = """
20         const infoboxes = document.querySelectorAll('.infobox, .
21         sidebar, .vertical-navbox, .navbox, .portal, .toc, .metadata, .
22         ambox, .hatnote, .shortdescription, .figcaption, .thumbcaption');
23         infoboxes.forEach(box => box.remove());
24         const figures = document.querySelectorAll('figure');
25         figures.forEach(f => f.remove());
26         """
27
28     css_list = ["h1#firstHeading", "#mw-content-text"]
29
30     # Configurazione del Crawler
31     crawler_config = CrawlerRunConfig(
32         cache_mode=CacheMode.BYPASS,
33         target_elements=css_list,
34         markdown_generator=md_generator,
35         js_code=remove_infobox_js
36     )
37
38     # Esecuzione del crawler da web o da HTML locale
39     async with AsyncWebCrawler(config=browser_config) as crawler:
40         if html_text == '':
41             result = await crawler.arun(url=url, config=
42             crawler_config)
43         else:
44             result = await crawler.arun(url=f"raw:{html_text}",
45             config=crawler_config)
46
47     # Pulizia del testo
48     clean_text = re.sub(r"[[[a-z]]|[[d+]]|[[edit]]|[[show]]|[[
49     update]]|[[\s*]]", "", result.markdown)
50     clean_text = re.sub(r'(\s*/_)(.*)\1', r'\2', clean_text)
51     clean_text = re.sub(r"[[citation needed]]|[[clarification needed
52     ]]|[[supporting]]|[[[A-Z]+]]", "", clean_text)
53     clean_text = re.sub(r"^\s*/_)(.*)\1", "", clean_text, flags=re
54     .MULTILINE)
55     clean_text = re.sub(r'^.*Main article[s]?:\s*.*$\n?', '',
56     clean_text, flags=re.MULTILINE | re.IGNORECASE)
57     clean_text = re.sub(r"\s*/_)(.*)\1", "", clean_text)
58     clean_text = re.sub(r"\s*/_)(.*)\1", "", clean_text)
59     text = re.split(r"##\s*(?:See also|References|Notes|Further
60     reading|External links)", clean_text, flags=re.IGNORECASE)
61     clean_text = text[0].strip()
62
63     # Creazione del JSON
64     title_match = re.search(r'<title>(.*?)</title>', result.html, re.
65     IGNORECASE)
66     page_title = title_match.group(1).replace(" - Wikipedia", "") if
67     title_match else "title_not_found"
68
69     return {
70         "url": url,
71         "domain": "en.wikipedia.org",
72         "title": page_title,
73         "html_text": result.html,
74         "parsed_text": clean_text
75     }

```

3.2.3 Motore di calcolo delle metriche

La valutazione quantitativa dell'accuratezza nell'estrazione delle informazioni operata dai *parser* è realizzata da un modulo indipendente. Questo componente, denominato `accuracy_tester.py`, funge da vero e proprio motore di calcolo: riceve in ingresso sia il testo pulito che il rispettivo testo di riferimento (*gold standard*), compilato accuratamente a mano. L'algoritmo converte i testi in due insiemi distinti di parole per il calcolo delle metriche tradizionali, elaborando contestualmente i testi interi per l'estrazione delle metriche semantiche vettoriali. Successivamente li confronta e restituisce un dizionario strutturato con le valutazioni complete, gestendo opportunamente tutti i casi limite e le eccezioni, come la ricezione di stringhe vuote o il verificarsi di divisioni per zero.

L'implementazione utilizza alcune librerie standard per l'elaborazione del linguaggio naturale e l'apprendimento automatico, tra cui *NLTK* per la metrica METEOR, *Scikit-learn* per la vettorializzazione TF-IDF e il calcolo della *Cosine similarity*, e *BERTScore* per la valutazione semantica. Un dettaglio rilevante riguarda proprio quest'ultima metrica: per rispettare i vincoli di memoria RAM dell'infrastruttura, il calcolo di BERTScore è stato realizzato con l'uso del modello alleggerito `distilbert-base-uncased`, così da garantire un ottimo compromesso tra accuratezza semantica e prestazioni computazionali.

Di seguito è riportato il cuore della logica algoritmica del modulo, omettendo per brevità i blocchi di gestione delle eccezioni e dei casi limite (come la ricezione di stringhe vuote).

Listing 3.3. Logica di estrazione dei token e calcolo delle metriche di valutazione.

```

1 import re
2 from nltk.translate.meteor_score import meteor_score
3 from sklearn.feature_extraction.text import TfidfVectorizer
4 from sklearn.metrics.pairwise import cosine_similarity
5 from bert_score import score
6
7 def calculate_metrics(parsed_text: str, gold_text: str) -> dict:
8
9     # ... [Omissioni: Gestione stringhe vuote e sanitizzazione] ...
10
11     # Estrazione dei token tramite regex
12     extracted_words = re.findall(r'\b\w+\b', parsed_text.lower())
13     gs_words = re.findall(r'\b\w+\b', gold_text.lower())
14     extracted_tokens = set(extracted_words)
15     gs_tokens = set(gs_words)
16
17     if not extracted_tokens or not gs_tokens:
18         return {
19             "precision": 0.0,
20             "recall": 0.0,
21             "f1": 0.0,
22             "jaccard_similarity": 0.0,
23             "bigram_overlap": 0.0,
24             "cosine_similarity": 0.0,
25             "meteor": 0.0,
26             "bert_score": 0.0
27         }
28

```

```

29     # Metriche tradizionali basate sugli insiemi
30     intersection = extracted_tokens & gs_tokens
31     union = extracted_tokens | gs_tokens
32
33     precision = len(intersection) / len(extracted_tokens)
34     recall = len(intersection) / len(gs_tokens)
35     f1 = (2 * precision * recall) / (precision + recall) if (
precision + recall) > 0 else 0.0
36
37     # Metriche avanzate
38     jaccard_similarity = len(intersection) / len(union) if len(union)
> 0 else 0.0
39
40     # Creazione di insiemi di coppie di parole consecutive per Bigram
overlap
41     ex_bigrams = set(zip(extracted_words[:-1], extracted_words[1:]))
42     gs_bigrams = set(zip(gs_words[:-1], gs_words[1:]))
43     bigram_overlap = len(ex_bigrams & gs_bigrams) / min(len(
ex_bigrams), len(gs_bigrams)) if ex_bigrams and gs_bigrams else
0.0
44
45     # Cosine similarity con vettorializzazione TF-IDF
46     vectorizer = TfidfVectorizer(token_pattern=r'(?u)\b\w+\b')
47     tfidf_matrix = vectorizer.fit_transform([gold_text, parsed_text])
48     cos_similarity = cosine_similarity(tfidf_matrix[0:1],
tfidf_matrix[1:2])[0][0]
49
50     # METEOR e BERTScore (modello distilbert per ottimizzazione RAM)
51     try:
52         meteor = meteor_score([gs_words], extracted_words)
53     except Exception:
54         meteor = 0.0
55
56     try:
57         P, R, F1_bert = score([parsed_text], [gold_text], lang="en",
model_type="distilbert-base-uncased", verbose=False)
58         bert_score = F1_bert.item()
59     except Exception as e:
60         print(f"Error BERTScore: {e}")
61         bert_score = 0.0
62
63     return {
64         "precision": float(precision),
65         "recall": float(recall),
66         "f1": float(f1),
67         "jaccard_similarity": float(jaccard_similarity),
68         "bigram_overlap": float(bigram_overlap),
69         "cosine_similarity": float(cos_similarity),
70         "meteor": float(meteor),
71         "bert_score": float(bert_score)
72     }

```

3.3 Progettazione del frontend

L'interfaccia utente, esposta sulla porta 8004, è stata progettata per fornire un punto di accesso intuitivo alle funzionalità del sistema. Si compone dei seguenti tre moduli principali:

- **Gestione delle richieste (`frontend.py`):** sviluppato anch'esso in FastAPI, questo modulo agisce come un'interfaccia tra l'utente e il backend. Utilizza il client asincrono `httpx` per comunicare internamente con i servizi del *backend*. Recupera dinamicamente la lista dei domini supportati e degli URL presenti nel database per popolare l'interfaccia all'avvio. Interroga i *gold standard* e monitora in tempo reale lo stato dei componenti (Backend, MariaDB e Ollama). Gestisce l'intero ciclo di richiesta-risposta che comprende tre flussi principali: l'analisi di URL singoli coordinando l'estrazione del testo per qualsiasi URL compatibile e, qualora la risorsa sia presente nel dataset di riferimento, innescando automaticamente il recupero e la visualizzazione delle metriche di valutazione; un pannello di amministrazione CRUD per la gestione interattiva dei dati di riferimento ed infine una *dashboard* statistica per monitorare facilmente i risultati.
- **Motore di rendering (`templates/`):** la presentazione dei dati è affidata a Jinja2 per la generazione dinamica di pagine HTML, stilizzate tramite CSS dedicato. Questo motore è fondamentale poiché consente di scrivere codice simile alla sintassi Python grazie a dei *placeholders* direttamente all'interno del documento, garantendo velocità ed espressività [8]. L'interfaccia è in grado di adattarsi ai dati ricevuti, mostra messaggi di errore contestuali e si articola in diverse aree specializzate: una *home page* di riepilogo dello stato, un'interfaccia di parser con tre colonne per facilitare il confronto testuale tra HTML grezzo, *gold standard* e testo estratto, esponendo contestualmente sotto le relative metriche dell'URL preso in input. Sono inoltre presenti un pannello per l'inserimento o rimozione di risorse web e una pagina analitica per vedere i risultati complessivi di ciascun dominio, sia metriche base e avanzate che giudizio dell'LLM.
- **Stile (`static/`):** questa directory contiene il foglio di stile CSS che definisce l'estetica dell'applicazione.

3.3.1 Architettura API e gestione delle latenze

Il cuore logico dell'interfaccia utente risiede nello script `frontend.py` che funge da *API gateway* tra le interazioni dell'utente e i servizi del *backend*, oltre a coordinare il caricamento dei file statici e dei *template* HTML tramite Jinja2. Dal punto di vista ingegneristico, una delle sfide principali di questo modulo riguarda la gestione delle latenze. Infatti, operazioni come l'estrazione del testo, il calcolo delle metriche e, soprattutto, la chiamata del modello linguistico (*LLM-as-a-Judge*), possono richiedere una grande quantità di tempo, persino minuti. Per evitare di bloccare il *thread* principale del server durante l'attesa delle risposte e continuare a garantire

il normale funzionamento del sistema, le comunicazioni HTTP interne sono state implementate in modo interamente asincrono sfruttando la libreria `httpx` e il paradigma `async/await`.

Di seguito è riportata una sintesi dell'architettura del modulo, focalizzata sulla rotta di analisi principale. Si può notare l'impostazione di un *timeout* esteso e l'uso di *context manager* asincroni per garantire una corretta chiusura delle connessioni di rete anche in caso di eccezioni.

Listing 3.4. Inizializzazione dell'applicazione ed esempio di gestione asincrona delle richieste nel frontend.

```

1  from fastapi import FastAPI, Request, Form
2  from fastapi.templating import Jinja2Templates
3  from fastapi.staticfiles import StaticFiles
4  import httpx
5
6  app = FastAPI(title="Frontend progetto")
7
8  # Montaggio risorse statiche e template
9  app.mount("/static", StaticFiles(directory="static"), name="static")
10 templates = Jinja2Templates(directory="templates")
11 BACKEND_URL = "http://backend:8003"
12
13 @app.post("/analyze")
14 async def analyze_url(request: Request, url: str = Form(...), mode:
15     str = Form("live")):
16     domains, gs_urls = await get_home_data()
17
18     parsed_data = None
19     eval_data = None
20     gold_text = None
21     judge_data = None
22     error = None
23     is_local = (mode == "local")
24
25     # Client HTTP asincrono con timeout esteso
26     async with httpx.AsyncClient(timeout=1800.0) as client:
27         try:
28             # Richiesta al backend per il parsing
29             parse_resp = await client.post(f"{BACKEND_URL}/parse",
30                 json={"url": url, "local": is_local})
31
32             if parse_resp.status_code == 200:
33                 parsed_data = parse_resp.json()
34
35             # Recupero del gold standard
36             gs_resp = await client.get(f"{BACKEND_URL}/
37                 gold_standard", params={"url": url})
38
39             if gs_resp.status_code == 200:
40                 gold_text = gs_resp.json().get("gold_text")
41                 # Calcolo delle metriche
42                 eval_resp = await client.post(f"{BACKEND_URL}/
43                     evaluate", json={
44                         "parsed_text": parsed_data["parsed_text"],
45                         "gold_text": gold_text
46                     })

```

```

43         if eval_resp.status_code == 200:
44             eval_data = eval_resp.json()
45
46             # Chiamata all'LLM
47             judge_resp = await client.post(f"{BACKEND_URL}/
evaluate_judge", json={
48                 "parsed_text": parsed_data["parsed_text"],
49                 "gold_text": gold_text
50             })
51             if judge_resp.status_code == 200:
52                 judge_data = judge_resp.json()
53
54             # Chiamata all'LLM reference-free
55             judge_no_gs_resp = await client.post(f"{
BACKEND_URL}/evaluate_judge_no_gs", json={
56                 "parsed_text": parsed_data["parsed_text"],
57                 "html_text": parsed_data["html_text"]
58             })
59             if judge_no_gs_resp.status_code == 200:
60                 judge_no_gs_data = judge_no_gs_resp.json()
61                 if judge_data is None:
62                     judge_data = {}
63
64                 judge_data["judge_score_no_gs"] =
judge_no_gs_data.get("judge_score")
65                 judge_data["judge_feedback_no_gs"] =
judge_no_gs_data.get("judge_feedback")
66             else:
67                 if judge_data is None:
68                     judge_data = {}
69                 judge_data["judge_score_no_gs"] = 0
70                 judge_data["judge_feedback_no_gs"] = f"ERRORE
DI SISTEMA: {judge_no_gs_resp.status_code} - {judge_no_gs_resp.
text}"
71             else:
72                 error = parse_resp.json().get("detail", f"Error in
backend: {parse_resp.status_code}")
73
74             except Exception as e:
75                 error = f"Error in communication with the server: {str(e)}
}"
76
77         # Immissione dei risultati nel motore Jinja2
78         return templates.TemplateResponse(
79             request=request,
80             name="analyze.html",
81             context={
82                 "domains": domains,
83                 "gs_urls": gs_urls,
84                 "parsed_data": parsed_data,
85                 "gold_text": gold_text,
86                 "eval_data": eval_data,
87                 "judge_data": judge_data,
88                 "error": error,
89                 "selected_mode": mode
90             }
91         )

```

3.4 Gestione dei dati

Per garantire la persistenza dei risultati e ottimizzare le risorse di rete, la *pipeline* integra un database relazionale MariaDB. Questa strategia di *caching* locale permette di memorizzare l'HTML delle pagine scansionate, evitando richieste HTTP ridondanti in caso di valutazioni successive sullo stesso URL.

Il database viene inizializzato al primo avvio popolando automaticamente le tabelle di sistema leggendo i file JSON contenuti nella directory `gs_data/`. Lo schema relazionale si articola nelle seguenti tabelle (le chiavi primarie sono evidenziate in grassetto):

- `web_resources` (**url**, domain, title, html_text, created_at): memorizza l'HTML grezzo prelevato durante la scansione, fungendo da anagrafica centrale.
- `gold_standard` (**url**, gold_text, created_at): archivia il testo pulito di riferimento necessario per il calcolo delle metriche di accuratezza. L'attributo `url` funge da chiave esterna.
- `parsed_pages` (**url**, parsed_text, created_at): conserva il testo elaborato e restituito dal *crawler*.
- `evaluation_results` (**url**, token_level_metrics, llm_judgments, evaluated_at): archivia i risultati finali. Al momento dell'inizializzazione i campi assumono valore NULL e vengono popolati dinamicamente. Le metriche insiemistiche vengono salvate in `token_level_metrics`, mentre il voto numerico e il *feedback* testuale dell'intelligenza artificiale risiedono nel campo `llm_judgments`.

Bibliografia

- [1] Patrick Lewis et al. «Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks». In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 9459–9474.
- [2] Amazon Web Services (AWS). *Cos'è la Retrieval-Augmented Generation (RAG)?* URL: <https://aws.amazon.com/it/what-is/retrieval-augmented-generation/> (visitato il giorno 31/08/2026).
- [3] Docker Inc. *Docker: Accelerated Container Application Development*. <https://www.docker.com/>. Ultimo accesso: 31 agosto 2026.
- [4] Sebastián Ramírez. *FastAPI*. <https://fastapi.tiangolo.com/>. Ultimo accesso: 1 settembre 2026.
- [5] Pydantic Services Inc. *Pydantic: Data validation using Python type hints*. <https://pydantic.dev/docs/validation/latest/get-started/>. Ultimo accesso: 1 settembre 2026.
- [6] Encode OSS. *HTTPX: A next-generation HTTP client for Python*. <https://www.python-httpx.org/>. Ultimo accesso: 1 settembre 2026.
- [7] Crawl4AI Contributors. *Crawl4AI: Open-source LLM Friendly Web Crawler & Scraper*. <https://crawl4ai.com/>. Ultimo accesso: 2 settembre 2026.
- [8] Pallets. *Jinja Documentation (Stable)*. <https://jinja.palletsprojects.com/en/stable/>. Ultimo accesso: 2 settembre 2026.